

Implementação do Perceptron: Avaliação de Desempenho em Problemas de Classificação Binária

Hebert Alan Kubis
Engenharia Mecatrônica
Universidade Federal de Santa Catarina (UFSC)
Centro Tecnológico de Joinville, Brasil

Resumo—Este documento apresenta a implementação em linguagem C de uma rede neural artificial do tipo Perceptron de camada única, desenvolvida para resolver problemas de classificação binária. O algoritmo base foi estruturado de forma genérica, logo não se limita à classificação unicamente binária. A implementação foi testada utilizando uma base de dados gerada sinteticamente para ser linearmente separável e a base de dados Iris, utilizando a metodologia hold-out para validação.

Index Terms—Perceptron, Inteligência Artificial, Classificação Binária, C, Python.

I. INTRODUÇÃO

O Perceptron é a forma mais simples de rede neural artificial, sendo composto por um único neurônio de saída, e é utilizado para resolver problemas de classificação linearmente separáveis.

O algoritmo desenvolvido neste documento implementa o perceptron para classificar dados em duas classes distintas, utilizando a função de ativação em degrau para determinar a saída binária, mas não se limita a isso, podendo ser utilizado para classificação multiclasse com a adição de mais neurônios na saída.

II. METODOLOGIA E CONFIGURAÇÃO DA REDE

A. Implementação do Motor Neural

O Perceptron foi implementado em C, baseando-se no paradigma de programação orientada a objetos, utilizando estruturas para representar os neurônios e, também, para representar a rede como um todo. A implementação inclui a inicialização dos pesos das entradas, o processo de treinamento utilizando o algoritmo de aprendizado supervisionado e a função de ativação para gerar as previsões.

B. Estratégia de Validação (Hold-out)

Para garantir a integridade da avaliação do modelo, adotou-se a metodologia de validação do tipo *Hold-out*. O conjunto de dados original foi segmentado de forma estática na proporção de 2/3 para o subconjunto de Treinamento e 1/3 para o subconjunto de Teste e embaralhado com a função `rand()` a cada época, sendo a seed redefinida a cada execução do programa.

As métricas de avaliação foram calculadas exclusivamente sobre os dados de teste que a rede nunca observou durante o processo de ajuste.

C. Parâmetros da Rede

Para o treinamento do Perceptron, foram definidos os seguintes parâmetros:

- **Taxa de Aprendizado (η):** 0.65
- **Critério de Parada:** Número máximo fixo de 100 *epochs*.
- **Função de Ativação:** Função Degrau (0 e 1).
- **Arquitetura:** 1 neurônio de saída.

III. TESTES E RESULTADOS COMPUTACIONAIS

A. Base de Dados Linearmente Separável (2D)

O primeiro experimento consistiu na aplicação da rede para segmentar uma base de dados sintética gerada de forma a garantir a separabilidade linear no plano bidimensional. O *dataset* foi balanceado, sendo composto por classes delimitadas em quadrantes opostos. Com esta base bidimensional aplicada ao Perceptron, a fronteira de decisão foi ajustada de forma simples e rápida (já na segunda *epoch*), como mostra a figura 1, onde a linha verde tracejada representa a fronteira de decisão ajustada pelo Perceptron.

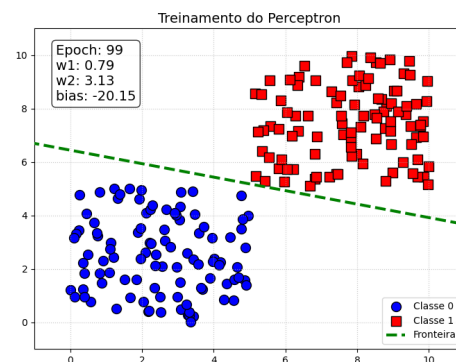


Figura 1. Fronteira de decisão do Perceptron ajustada no plano bidimensional.

B. Desempenho na base linearmente separável

Após a execução do algoritmo de treinamento e predição, foram extraídas métricas de avaliação via linguagem Python utilizando a biblioteca Scikit-learn [1] e para visualização foi utilizada a biblioteca Matplotlib [2].

Para este teste, não será mostrado o gráfico de acurácia x época, pois a acurácia atingiu 100% já na segunda época. De

tudo modo, a matriz de confusão é apresentada na figura 2, onde pode-se observar que o modelo classificou corretamente todas as amostras, sem apresentar Falsos Positivos ou Falsos Negativos.

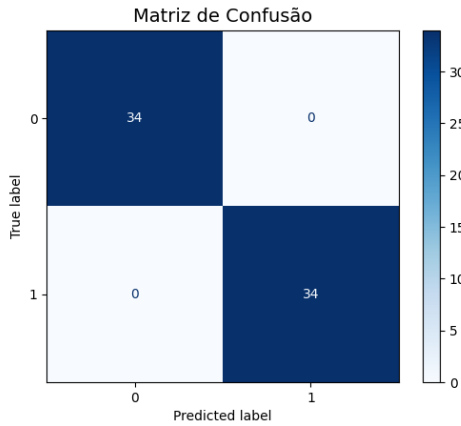


Figura 2. Matriz de Confusão para a base linearmente separável.

C. Base de Dados Iris (Versicolor x Virginica)

O segundo teste foi realizado na base de dados Iris, utilizando apenas as classes Versicolor e Virginica, classificadas, respectivamente, como 0 e 1, que não são linearmente separáveis, para verificar o comportamento do Perceptron em um cenário onde é impossível convergir. Nesse teste também foi tratada a questão da multidimensionalidade, utilizando as quatro características disponíveis para o treinamento e predição.

D. Desempenho na base Iris

Como a base é multidimensional, os dados analisados não podem ser visualizados de forma espacial direta. Deste modo, o Perceptron será avaliado com base na curva de Acurácia x Época e na matriz de confusão.

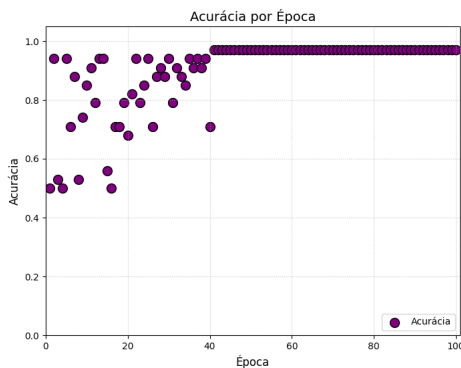


Figura 3. Curva de Acurácia ao longo das épocas para a base Iris.

Na figura 3 é possível visualizar que a acurácia do modelo oscilou bastante durante as primeiras épocas (provavelmente devido à taxa de aprendizado alta utilizada). Contudo, ao atingir o limite de iterações, a precisão da última época computada

resultou em 97%. É importante destacar que o comportamento esperado para dados não linearmente separáveis é que o modelo oscile indefinidamente, visto que o erro de classificação nunca é zerado e a regra de aprendizado continua forçando atualizações nos pesos.

Neste caso específico, a estabilização da curva de acurácia em um patamar alto ocorreu porque os contínuos ajustes da fronteira de decisão não foram grandes o suficiente para alterar a predição das amostras do conjunto de teste. Vale ressaltar que o obstáculo para atingir 100% de acurácia é a limitação matemática fundamental do Perceptron de camada única, que é incapaz de traçar fronteiras de decisão não lineares para contornar a zona de sobreposição entre a Versicolor e a Virginica.

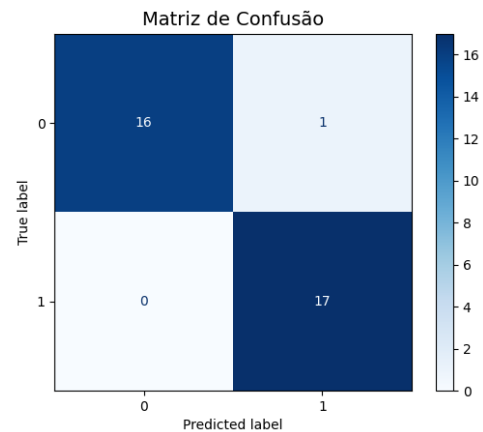


Figura 4. Matriz de Confusão para a base Iris (Versicolor x Virginica).

Outra métrica importante para avaliar o desempenho do modelo é a matriz de confusão, apresentada na figura 4. Nela é possível observar que o modelo classificou uma amostra como Virginica quando na verdade era Versicolor, o que corresponde a um Falso Positivo. Este resultado é coerente com a acurácia de 97% apresentada na curva de acurácia x época, indicando que o modelo teve um desempenho elevado, mas não perfeito, dadas as limitações estruturais da rede frente à sobreposição das classes.

IV. CONCLUSÃO

O perceptron implementado em C funcionou conforme o esperado, conseguindo classificar corretamente os dados da base linearmente separável e não convergindo matematicamente para uma solução estática na base Iris. A rede demonstrou o comportamento teórico esperado de oscilação em cenários não separáveis, embora o critério de parada por épocas tenha garantido uma fronteira de decisão com alta taxa de acerto.

REFERÊNCIAS

- [1] sklearn.metrics. Scikit-learn, 2026. Disponível em: <https://scikit-learn.org/stable/api/sklearn.metrics.html>. Acesso em: 22 de maio de 2026.
- [2] matplotlib.pyplot. Matplotlib, 2026. Disponível em: <https://matplotlib.org/stable/gallery/pyplots/index.html>. Acesso em: 22 de maio de 2026.
- [3] Material de aula do curso de Sistemas Inteligentes, ministrado pelo professor Dr. Pablo Andretta Jaskowiak.